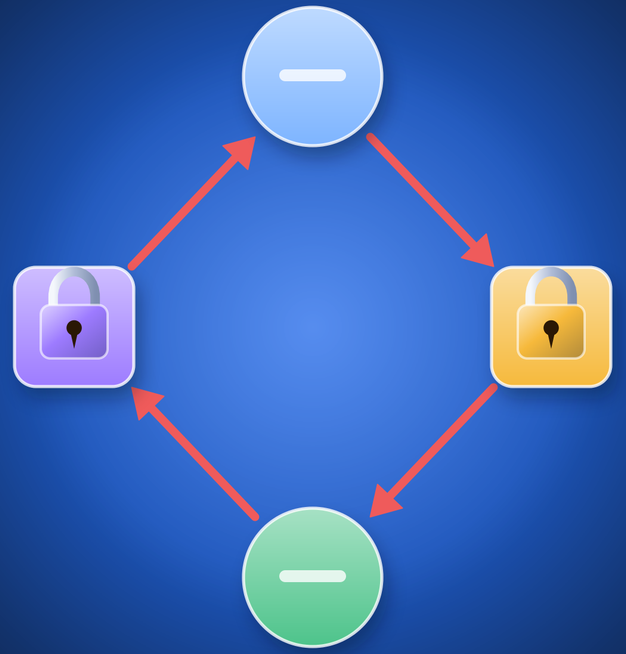
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 9

Deadlock, Lock Order and Recovery

Lab 9 guide: make two workers wait for each other, draw the cycle, and remove it with one lock order.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Draw holdings and requests that form a circular wait.
2. Reproduce a two-worker conflict with bounded waits, and read how a timeout recovers from it.
3. Apply one global lock order, and explain why it removes the cycle.

THIS LAB

Lab 9

First the concepts, with pictures. Then the lab on the server. Wait traces and the site-to-site exercise are optional extensions, so they come last.

DISCUSSION

Two people each hold one key and want the other's. What happens?

Neither will give theirs up first. How long will they wait? How could they have avoided it?

CONCEPTS FIRST

Holding, waiting and cycles

Three ideas: a worker can hold one resource while it asks for another, waits can form a circle, and a rule or a timeout can break it.

Four conditions for a deadlock

01

Mutual exclusion

A resource can be used by only one worker at a time.

02

Hold and wait

A worker holds one resource while it asks for another.

03

No preemption

A resource cannot be taken away. Its holder must release it.

04

Circular wait

Worker A waits for B, and B waits for A: the waits form a circle.

All four must hold together. Removing any one of them prevents a deadlock. This lab removes the fourth.

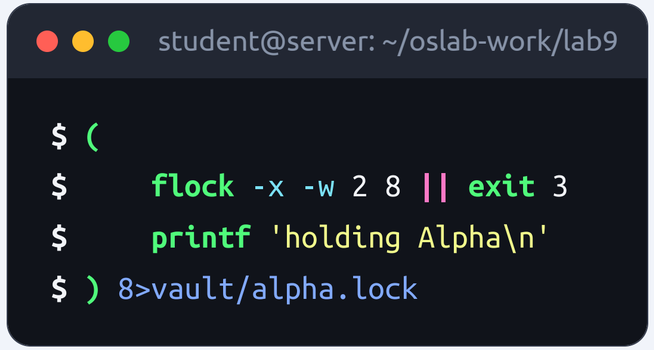
Draw it as holdings and requests

Worker	Holds	Requests
A	Alpha	Beta
B	Beta	Alpha

Draw an arrow from a resource to the worker that holds it, and from a worker to the resource it waits for.

Here: Alpha to A, A to Beta, Beta to B, B to Alpha. The arrows close a circle, so neither worker can go on.

A lock with a time limit



A data file holds the data. A lock file stands for cooperative exclusive access. Creating a file named locked is not a lock.

flock -x -w 2 waits at most two seconds for the lock. If it does not get it, the status is not zero, and the script can stop waiting.

Descriptor 8 stays open until the group of commands ends, and that is when the lock is released.

Two ways to deal with a deadlock

BEFORE IT HAPPENS

Prevention: a global lock order

Every worker takes Alpha before Beta. A worker that waits for Alpha holds nothing yet, so no circle can form.

AFTER IT HAPPENS

Recovery: a timeout

A worker that waits too long gives up and releases what it holds. The others can then go on.

IN PRACTICE

Use both

Ordering stops this circle. Timeouts also bound waits that come from other causes. Neither gives fairness.

LAB 9

The vault that freezes

QuantumTech's Alpha and Beta recovery vaults freeze during a drill. Model them, watch the wait, and repair the order.

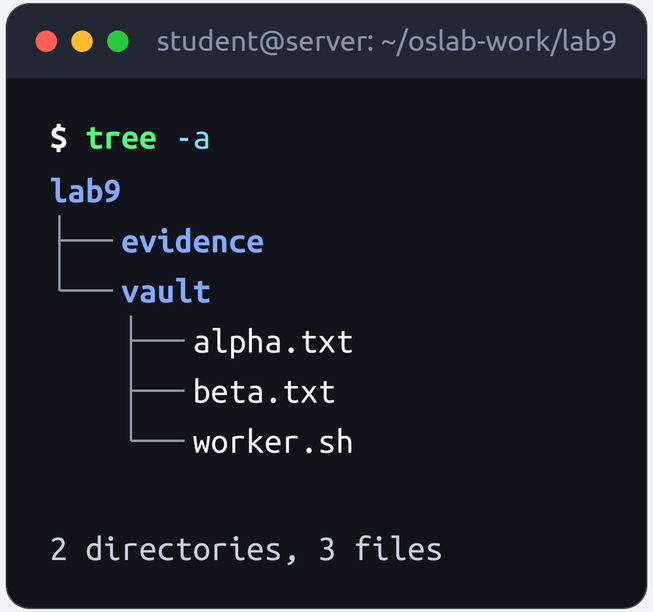
The scenario

SCENARIO

QuantumTech's recovery vaults

Alpha and Beta freeze during a synchronization drill. Model them with two lock files you own, observe a bounded wait cycle, and repair the acquisition order.

Every wait has a time limit. Nothing runs for ever, and you never delete a lock file while a worker uses it.



Inside the 120 minutes

Minutes	What happens
0 to 10	Introduction, objectives and setup
10 to 25	Level 1, guided: locks and the vault
25 to 35	Written prediction
35 to 55	Levels 2 and 3: naive workers, local deadlock

Minutes	What happens
55 to 70	Levels 5 and 6: global order, timeout recovery
70 to 85	Tests and feedback
85 to 100	Individual changed-case checkpoint
100 to 120	Correction, cleanup and submission

How you work

Your own account

Log in with your own account. Use only your own files and processes. Use the saved PIDs and bounded waits. Never use pkill.

AI tools

Optional in the investigation and the tests. Not allowed in the prediction and the individual checkpoint.

Evidence

Keep what you observe in the evidence folder. Short, attributable evidence earns more than long output.

Initial prediction accuracy is lightly weighted. A reasoned attempt and an evidence-based correction earn credit.

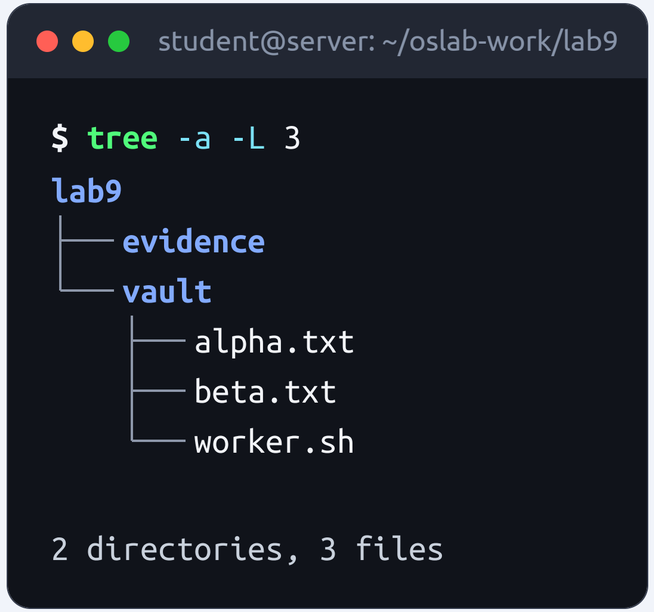
Setup: start the lab

student@server: ~

```
$ whoami
$ command -v oslab
$ oslab doctor
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab9
$ cd "$OSLAB_WORKSPACE/lab9"
$ pwd
$ mkdir -p evidence
$ find . -maxdepth 3 -type f
```

- whoami shows your account. command -v oslab and oslab doctor check the helper.
- If the helper is missing, follow the local setup guide in the course repository.
- oslab start lab9 makes the lab folder. It keeps your work if you start again, and it does not change your current folder, so cd into it yourself.
- If you are resuming, look at the existing files before you run a command that writes to them.

Compare with the starting tree



vault/alpha.txt and beta.txt hold the data of the two vaults.

vault/worker.sh is an editable starter skeleton.

The helper also writes a hidden file, .oslab-managed.json. Leave it as it is.

GUIDED

Locks and the vault

Level 1, with the instructor: fifteen minutes.

Step 1 and 2: look, then take one lock

student@server: ~/oslab-work/lab9

```
$ cat vault/alpha.txt vault/beta.txt
$ mkdir -p vault/coord
$ (
$     flock -x -w 2 8 || exit 3
$     printf 'holding Alpha\n'
$ ) 8>vault/alpha.lock
```

The text files are data. The .lock files stand for exclusive access.

Descriptor 8 is open until the group ends. A real program would keep the protected work inside this group.

Step 3: one lock, two workers

student@server: ~/oslab-work/lab9

```
$ (
$     flock -x -w 2 8 || exit 3
$     sleep 3
$ ) 8>vault/alpha.lock &
$ holder=$!
$ sleep 0.2
$ flock -x -w 1 vault/alpha.lock true
$ printf 'contender status=%s\n' "$?"
$ wait "$holder"
```

The first worker holds Alpha for three seconds. The second asks for it and gives up after one.

Expect a timeout if the holder got the lock first. The short sleep helps but does not prove it.

THINK

Observe

What is held while another worker waits? Does a timeout prevent a deadlock, or only let a worker stop waiting?

PREDICT

Guess first, then try

Before you run the experiment. Ten minutes.

The prediction

PREDICTION

Write this down first

A holds Alpha and requests Beta. B holds Beta and requests Alpha. Draw both wait edges, and predict whether either can get its second resource before someone releases a lock.

HOW

The rules

Alone, and without AI. On paper or in the course worksheet, before you run the command. You may compare with a neighbour for five minutes, but keep your own answer.

A wrong prediction costs nothing. It is marked for being made and for the evidence-based correction afterwards.

LEVELS 2 AND 3

Naive workers and a local deadlock

Twenty minutes, from minute 35 to 55. Work alone.

Step 1: the starting worker, part 1

vault/worker.sh (1 of 3)

```
#!/usr/bin/env bash
set -euo pipefail
[[ $# -eq 2 && "$1" =~ ^(A|B)$ && "$2" =~ ^(opposite|ordered)$ ]] || exit 2
role=$1
mode=$2
vault=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)
first=alpha
second=beta
# TODO: in ordered mode, every worker must use one global order.
if [[ "$role" = B ]]; then first=beta; second=alpha; fi
exec 8>"$vault/$first.lock"
flock -x -w 2 8 || { echo "$role first lock timeout"; exit 3; }
printf '%s holds %s\n' "$role" "$first"
```

Save the whole script as vault/worker.sh with the three parts on this and the next two slides. B reverses the order by default: that is the TODO you will fix.

set -euo pipefail: stop on an error (-e), on an unset variable (-u), and when any command in a pipeline fails (pipefail).

The vault= line finds the script's own folder: BASH_SOURCE[0] is its path, dirname drops the file name, pwd -P prints the real folder.

Each flock waits two seconds. Each failure exits with its own status: 3, 4 or 5.

Step 1: the barrier, part 2

vault/worker.sh (2 of 3)

```
if [[ "$mode" = opposite ]]; then
    touch "$vault/coord/$role.ready"
    other=B
    [[ "$role" = B ]] && other=A
    for step in {1..30}; do
        [[ -f "$vault/coord/$other.ready" ]] && break
        sleep 0.1
    done
    [[ -f "$vault/coord/$other.ready" ]] || { echo 'barrier timeout'; exit 4; }
fi
```

The barrier is a teaching instrument. Each worker says it is ready, then waits for the other to say so.

It makes sure both workers hold their first lock before either asks for its second.

In ordered mode it is skipped: one worker cannot take Alpha while the other holds it.

Step 1: the second lock, part 3

vault/worker.sh (3 of 3)

```
exec 9>"$vault/$second.lock"
printf '%s requests %s\n' "$role" "$second"
flock -x -w 2 9 || { echo "$role second lock timeout"; exit 5; }
printf '%s holds both\n' "$role"
```

After the barrier, each worker asks for its second lock, on descriptor 9, and reports what it holds.

SYNTAX

Why exec 8> and exec 9>?

exec with only a redirection opens the file for the rest of the script, so the lock lasts until the script ends.

Step 2: two workers, opposite order

student@server: ~/oslab-work/lab9

```
$ rm -f -- vault/coord/A.ready vault/coord/B.ready
$ timeout 6 bash vault/worker.sh A opposite > evidence/opposite-a.txt 2>&1 &
$ worker_a=$!
$ timeout 6 bash vault/worker.sh B opposite > evidence/opposite-b.txt 2>&1 &
$ worker_b=$!
$ if wait "$worker_a"; then rc_a=0; else rc_a=$?; fi
$ if wait "$worker_b"; then rc_b=0; else rc_b=$?; fi
$ printf 'A=%s B=%s\n' "$rc_a" "$rc_b"
$ cat evidence/opposite-a.txt evidence/opposite-b.txt
```

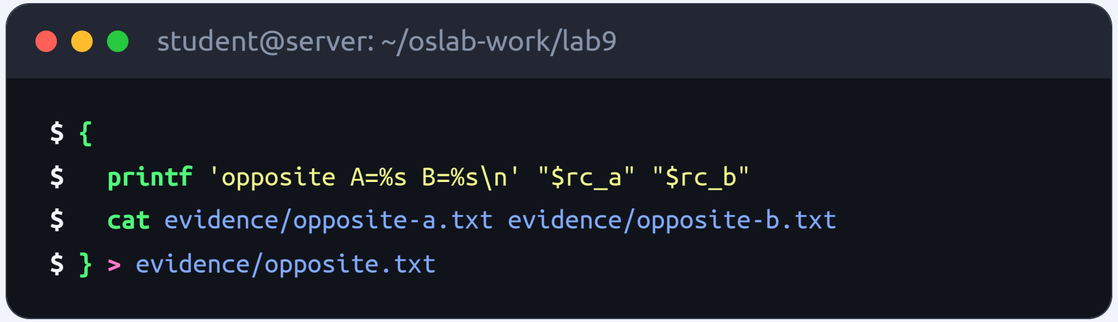
Clear only the two old ready markers, then start exactly two workers. timeout 6 stops a worker that waits too long.

The if wait ... construct keeps the status even when it is not zero. A status of 124 means timeout itself stopped the worker.

Step 3: draw it, and explain the recovery

What to do

1. Draw the holdings and requests at the barrier.
2. At least one worker should time out on the second lock.
3. Once it exits, it releases its first lock. The other worker may then finish instead of timing out too.
4. Explain that recovery. Do not claim that both must fail.



LEVELS 5 AND 6

Global order and timeout recovery

Fifteen minutes, from minute 55 to 70.

Step 1: one order for everyone

Edit the TODO

- In ordered mode, every worker takes Alpha first, then Beta.
- Opposite mode stays as it is, so you can reproduce the conflict.
- Leave the barrier in opposite mode only.

YOUR TURN

You write it

Nobody gives you the lines. Ask: which line decides first and second, and for which workers?

Step 2: run the ordered case

student@server: ~/oslab-work/lab9

```
$ timeout 6 bash vault/worker.sh A ordered > evidence/ordered-a.txt 2>&1 &
$ worker_a=$!
$ timeout 6 bash vault/worker.sh B ordered > evidence/ordered-b.txt 2>&1 &
$ worker_b=$!
$ if wait "$worker_a"; then rc_a=0; else rc_a=$?; fi
$ if wait "$worker_b"; then rc_b=0; else rc_b=$?; fi
$ {
$   printf 'ordered A=%s B=%s\n' "$rc_a" "$rc_b"
$   cat evidence/ordered-a.txt evidence/ordered-b.txt
$ } > evidence/ordered.txt
```

Save separate logs and both statuses. Explain why a worker that waits for Alpha cannot also hold Beta.

Keep the timeouts. Ordering prevents this circle. Timeouts also bound waits from other causes. Neither gives fairness, and neither promises that a larger system is free of deadlock.

Complete when, and hints

DONE

Complete when

Your logs show the wait cycle in the first case. Both ordered workers finish. The global rule is applied the same way everywhere.

Hints, one at a time

1. Draw held and requested resources.
2. Read the first-lock logs and the ready markers.
3. Reverse B only in the teaching opposite mode, and skip the barrier in ordered mode.

If AI suggests a repair, check that it changes both workers' policy and adds no new barrier wait.

Three cases, and what each proves

Fifteen minutes, from minute 70 to 85.

The three cases

Case	Intended evidence
Opposite, two workers	Both hold distinct first locks. The second lock conflicts, and a timeout recovers
Ordered, two workers	Both complete, one after the other, through Alpha
One opposite worker alone	The barrier times out within three seconds, which explains the missing participant

Status 124 means the outer timeout fired. Do not call an unexplained timeout a recovery. One lucky run does not prove that opposite order is safe. Repeat the opposite case only after you delete the two ready markers.

If something does not work

Old ready markers

Reusing them makes the barrier pass when it should not. Clear only A.ready and B.ready.

Lock files stay

Never delete an active lock file. Its existence is normal.

No pkill

Use the saved PIDs and the bounded waits. See man flock and the deadlock visualization.

INDIVIDUAL CHECKPOINT

Show what you can do alone

Fifteen minutes, from minute 85 to 100. No AI, no neighbours.

Why there is a checkpoint

It checks you, not the tools

Everything before it may use AI and help. The checkpoint shows what you understood without them.

A short question on paper

You answer one question on paper or on the course worksheet. Your earlier work does not need to be finished.

Practice question

The question on the next slide is public practice. A graded session may use a fresh private variant.

The question

CHECKPOINT

Practice question

A third worker needs only Beta. Must it also take Alpha to follow the global-order policy? Explain. Separately, say whether the timeout is prevention or recovery in the opposite-order example.

The instructor collects your answer before discussing the public key.

What to give

1. The result or diagnosis.
2. The mechanism.
3. One observation that could check it.

Look back, then hand in

Ten minutes of correction, ten to clean up and submit.

Correct your prediction

1. Keep your original prediction in view. Under it, write confirmed or corrected.
2. Cite the test that shows it.
3. Explain the OS mechanism in three to five sentences.

ANSWER

Three questions

(a) Which pair of edges formed the cycle? (b) Why must the teaching barrier be absent in ordered mode? (c) How can one timeout let the other worker finish?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab9/evidence"
$ cd "$OSLAB_WORKSPACE/lab9"
$ cp -- vault/worker.sh "$SUBMISSION_REPO/lab9/"
$ cp -- evidence/opposite.txt evidence/ordered.txt "$SUBMISSION_REPO/lab9/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

opposite.txt should hold the holdings and requests, the statuses, the recovery and your wait diagram. Cleanup is part of the lab: leave no worker running.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab9
├── evidence
│   ├── opposite.txt
│   └── ordered.txt
├── README.md
└── worker.sh

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

How it is marked: 10 points

Evidence for the lab objectives	Points
A bounded conflict reproduced, and one consistent global order	3
The opposite and ordered runs, and the missing-participant diagnosis	2
Circular wait, the barrier and timeout recovery explained; original prediction and correction	2
Individual checkpoint: result or diagnosis, and mechanism	2
Concise, attributable evidence and the required artifacts	1

Other valid commands and names earn credit. A naming difference costs points only when it breaks the result. If you use AI, note one useful suggestion and how you verified it.

Common mistakes

Mistake	What you see	Fix
Claiming both workers must time out	One finishes after the other gives up	Explain the recovery event
The barrier left in ordered mode	A worker waits for a partner that cannot arrive	Skip the barrier when ordered
Removing the timeouts after ordering	A future wait with no limit	Keep every flock bounded
Reusing the ready markers	A false coordination condition	rm only A.ready and B.ready first

BEYOND THE CORE

Wait traces and site-to-site sync

Optional extensions. They stay bounded, and they use only a narrow folder the instructor prepared.

Looking at who waits

student@server: ~/oslab-work/lab9

```
$ timeout 6 bash vault/worker.sh A opposite &
$ ps -o pid,ppid,stat,wchan:20,comm -p $!
$ wait
```

While a worker waits for a lock, ps shows it asleep, state S. The wchan column names the kernel place where it sleeps.

Take one sample, and say what you saw. A late sample may miss the wait entirely.

Site-to-site synchronization

CONSENT

Consent first

Two people agree on a narrow folder the instructor prepared. Nobody writes into another person's home.

ORDER

Same order on both sides

Both sites must take their locks in the same global order, or the circle comes back across machines.

SAFE

Bounded and logged

Every wait has a limit, and each side keeps a log of what it held and asked for.

Key takeaways

01

Four conditions

Mutual exclusion, hold and wait, no preemption and circular wait. Break one.

02

Draw the edges

Holdings and requests that close a circle are a deadlock.

03

Order prevents, timeout recovers

One global lock order stops this circle. A timeout only ends a wait that has started.

04

Keep every wait bounded

Even with an order, a limit stops a hidden problem from hanging for ever.

Next session

COMING UP

Lab 10: backups, archives and cron

Make a backup you can restore, and schedule it to run by itself.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 8, Deadlocks.